Final Video Link:
https://drive.google.com/file/d/1ClboeNPV0FUzKJtuAiVFYe_ZGqaJWpLm/view?usp=sharing

Project Report:

Terrain – National Park Adventure Planner

Team: Hetvi Patel, Aditi Shukla, Devanshi Sawhney, Manan Vora

1. Project Direction Changes

Throughout the project creation process, our team encountered several challenges that led to changes in the direction of our original proposal. While we maintained the core purpose of the project, creating a centralized hub for National Park trip planning, we adapted several of our original ideas to better fit technical efficiency and frontend backend separation.

Originally, we wanted to handle all external API calls for weather and air quality on the backend. The plan was to have our server aggregate data from OpenWeatherMap and AirNow APIs before sending it to the frontend. However, during implementation, we realized that this approach created unnecessary complexity. Instead, we decided to focus solely on serving persistent application data (parks, users, trips) and improved responsiveness.

Another major pivot involved our Data Sourcing Strategy. We used data from online datasets, but quickly realized those were not detailed enough so we had to do our own research and get more data from the NPS. This ensured our park information was comprehensive and could be updated programmatically. We complemented this with a large dataset of simulated users and reviews to stress test our application's features.

Finally, we changed our development timeline for the creative components. Our interactive map (using Leaflet/OpenStreetMap) was a success but the final dashboard for visualizing the integrated data feeds (weather, AQI, NPS alerts) on a single park details page was not fully completed. We decided that it was more important to ensure that our core CRUD functionalities for users and trip plans worked perfectly, and leave the unified dashboard for future work. We also created the dynamically changing park popularity and most active parks leaderboard, which actively changed based on what users across the platform were doing. This allowed us to deliver a stable, fully featured application for user management, trip organization, and community building.

2. Achievements and Failures

Achievements

Despite the changes, our final application successfully met many of the core goals outlined in the original proposal:

- **Consolidated Park Information:** We built a functional database and API that serves detailed park data, including descriptions, locations, and associated activities.
- **Secure User System:** We implemented full user authentication (login/signup) and authorization, ensuring users can only manage their own trip plans and data.
- **Interactive Trip Planner:** Users can create, read, update, and delete personalized trip plans, adding parks and personal notes to each.
- **Advanced Discovery Features:** We added a search endpoint using a stored procedure that lets users filter parks by keywords and activity types.
- **Scalable Data Foundation:** We successfully populated the database with 1,000 simulated users, reviews, and notes, proving the application can handle substantial data loads.
- **Community Building:** Dynamically changing park popularity and most active parks leaderboard

Failures
However, we were not able to fully build out one key feature:

- **Unified Park Dashboard:** The visualization layer that would combine real time weather, air quality, and NPS alerts on a single park details page was not fully integrated into the project. We believe its absence is the most significant gap, as it was intended to be the "one stop shop" insight for trip planners.

3. **Schema / Data Source Changes**
We started with a basic idea for our database, but when we actually built it, we had to make some key moves to make it tougher and smarter. We didn't change the main tables (like Users or Parks), but we made the rules much stricter. The biggest change was creating Composite Primary Keys, using both the trip ID and the park ID together on the TripPlanParks table. This is made so the database itself now guarantees you can't add the same park twice to one trip, which is way safer than hoping our code always catches it. Plus, when someone deletes their account, we put the whole cleanup process inside a database transaction. This makes sure all their trip plans and park links are wiped out perfectly, all at once, so we don't end up with useless, floating data. Finally, for the more difficult job of creating a trip summary, like counting unique states or figuring out if it's a "Local" or "Multi Region" adventure. We built a Stored Procedure called GenerateTripSummary right into the database. This is useful because the database is better and faster at running that kind of complex logic than our main application is. Ultimately, the final design is just better because it shifts the heavy lifting and critical rule enforcement to MySQL, making the whole system more reliable and easier for us to maintain long term.

4. Diagram Changes

The changes in our database design required transitioning from the logical structure presented in our UML diagram to functional code. The key changes were the implementation of Composite Primary Keys. such as (trip_plan_id, park_id} on the {TripPlanParks} table, which guarantee uniqueness, preventing data redundancy where the original UML only logically showed the relationship. Furthermore, we introduced the GenerateTripSummary Stored Procedure into the schema to move complex business logic, including state counting and trip classification, directly into the MySQL engine. This choice was so that the final design is more suitable because it delegates critical tasks like strict rule enforcement and heavy calculation to the database, ensuring greater reliability, consistency, and performance compared to relying solely on the application layer.

5. Functionalities Added and Removed

Functionalities Added:

To meet advanced database requirements and improve the user experience, we added several key features:

- **Database Transaction for User Deletion:** When a user is deleted, we remove their account and all related data (trip plans, reviews) inside a single transaction. This guarantees ACID compliance, ensuring data consistency is never left in a partial state if an error occurs.
- **Analytics Endpoints:** We created new API endpoints like /api/stats/popular-parks that execute aggregation queries. This gives us valuable insights, such as identifying the most saved parks, which can help with feature development.

Functionalities Removed:

We did not remove any core user facing functionality from the original proposal. The only significant change was the architectural decision: we didn't implement the real time weather updating functionality. As explained earlier this was due to how our app was implemented and due to the time constraints.

6. Advanced Database Project Complementation

The Terrain application relies on advanced database programs, specifically stored procedures and transactions, to enhance functionality, ensure data integrity, and improve performance. Here's how each component contributes:

1. GenerateTripSummary Stored Procedure

This stored procedure directly supports the Trip Planner feature by generating a human readable summary for any given trip plan.

Functionality:
It takes a trip_plan_id as input and outputs a formatted summary text that includes:

- The number of regions (states) covered in the trip.
- A list of parks included in the trip.
- A classification of the trip as either "Multi Region Adventure" or "Local Explorer".

How It Complements the App:

- Instead of showing raw data, the app can display a clean, descriptive summary of each trip plan, making it easier for users to quickly understand the scope of their adventure.
- The logic for determining trip type and formatting the summary is stored in the database, ensuring consistency across all frontend and backend calls.
- By executing this logic on the database server, we reduce the amount of data transferred and processing required on the application server.

3. Database Transaction for User Deletion
This ensures atomic deletion of a user and all associated data (trip plans, reviews, etc.).

How It Complements the App:

- Prevents lost records and ensures the database remains in a consistent state.
- Users can confidently delete their accounts knowing all personal data is completely removed.

7. Technical Challenges Encountered

Team Member: Aditi Shukla

Problem: Initially, fetching a list of parks and their associated activities resulted in inefficient "N+1" queries: one query to get all parks, then an additional query for each park to get its activities.
The initial backend logic fetched parks first, then looped through each result to fetch its activities in a separate database call.Future teams should plan their queries using JOINs from the start. Use a single query that JOINs the Parks and Activities tables (through the ParkActivities junction table) and uses GROUP_CONCAT() to combine all activity names into one field per park. This cuts the number of database calls from N+1to just 1.

Team Member: Hetvi Patel

Problem: The cascade deletion of a user and all their related data (trip plans, junction table entries, reviews) could fail if any one of the DELETE statements broke, leaving behind incomplete or inconsistent data in the database.

The delete operation was implemented as multiple sequential SQL statements without a mechanism to roll back all changes if one failed. Always use database transactions for multi step write operations. Start a transaction, run all your individual DELETE statements, and then COMMIT inside a try catch block. This ensures the operation is atomic and is essential for keeping the database in a consistent, clean state.

Team Member: Manan Vora

Problem: On the park details page, handling the state for several separate asynchronous calls (first fetching internal park data) made the UI harder to manage. The page ended up re rendering multiple times, showing partial data, or quietly failing when one of the APIs was slow.

Using separate useEffect hooks and useState variables for each data source created race conditions and complicated loading/error handling logic. Consolidate asynchronous data fetching logic by, using a state machine approach (e.g., { loading, error, data }) or a library like React Query or SWR designed for this purpose. These tools handle caching, duplication of requests, and provide a consistent way for loading and error states.

Team Member: Devanshi Swahney

Problem: Making the Leaflet map interactive (clicking a park marker) and syncing its state with the rest of the React application (updating a selected park sidebar/details panel) was challenging. Event handlers from the map library needed to update React state, which sometimes caused updates to be missed or the UI to fall out of sync.

Imperative map library events were not seamlessly integrating with React's state model. State updates triggered by map clicks were not always causing a re render of sibling components. When integrating third party libraries with React, lift the state up. Store the “selected park” in a parent component as the single source of truth. Pass a callback down to the map so that when a

marker is clicked, it calls setSelectedPark(). This ensures React manages all state changes. Also, consider using a ref to access the Leaflet map instance directly only when necessary.

8. Future Work

There are several areas where the Terrain application could continue to improve beyond the current version:

- Complete the Unified Dashboard: The top priority is finishing the integration and frontend presentation of real time weather, air quality index, and NPS alerts on the individual park details page. This was the key feature we wanted to bring to trip planners.
- Query Performance Optimization: We want to refactor the park and activities fetch to remove the N+1 query pattern by using a more optimized JOIN. We also plan to look into database triggers to automatically manage updated_at timestamps.
- Enhanced Discovery: Introduce more advanced filtering (by region, park type, best time to visit) and personalized park recommendations based on a user's saved trips or activity preferences.
- Social Features: Allow users to make trip plans public, share them with friends, or leave reviews on specific parks.

9. Division of Labor and Teamwork

The division of labor throughout the project was collaborative and adapted to the needs of each stage. All team members actively contributed to design discussions, debugging, and integration efforts.

Manan Vora took primary responsibility for the backend architecture, setting up the Express server, designing the core database schema, and building the API routes for parks and activities, including the integration with the NPS API.

Hetvi Patel focused on user centric backend functionality, implementing secure user authentication (login/signup), authorization logic, and all CRUD API routes for user profiles and trip plans, including the transactional delete feature.

Aditi Shukla led the frontend development, translating design mockups into a functional React application, building the UI components (trip planner, park cards), and managing the client side state.

Devanshi Swahney drove the creative component integration, implementing the interactive map using Leaflet and managing the frontend logic for fetching and displaying data from the OpenWeatherMap and AirNow APIs.

Backend and frontend members worked closely to define API contracts, and all team members contributed to troubleshooting and writing the final project report. We used regular meetings and a shared task list to track progress, ensuring the workload was balanced and that every major feature had oversight from multiple team members. This collaborative approach kept the project on track and resulted in a cohesive final product.